C++ Programming Textbook Notes

Module: Function Overloading in C++ | Compile-Time Polymorphism & Name Mangling

Author: Gaurav Kandpal **Platform:** CS-Simplified **Level:** Beginner to Advanced

1. POLYMORPHISM & FUNCTION OVERLOADING FOUNDATIONS

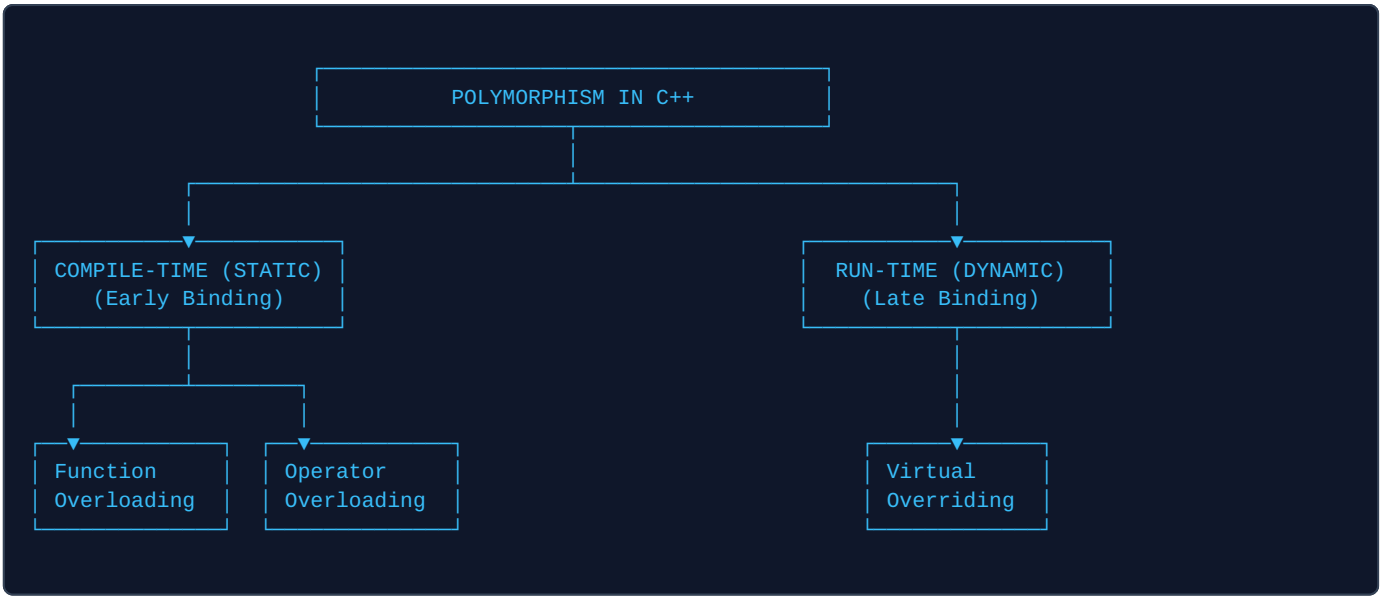
Polymorphism in OOP

C++ is an Object-Oriented Programming (OOP) language providing rich features to implement core OOP principles. One of the central pillars of OOP is **Polymorphism**.

Derived from Greek words (*poly* = many, *morph* = forms), **Polymorphism** refers to the ability of an entity (such as a function or operator) to take on multiple forms and exhibit different behaviors depending on the context or data types involved. It enables developers to write flexible, extensible, and clean code.

Types of Polymorphism in C++

- **1. Compile-Time Polymorphism (Static / Early Binding):** The exact function or operator to execute is determined at compile-time by the compiler based on argument types and signatures.
  - **Function Overloading:** Using the same function name with different parameter signatures.
  - **Operator Overloading:** Redefining standard C++ operators (e.g., +, <<) for custom user-defined classes.
- **2. Run-Time Polymorphism (Dynamic / Late Binding):** The function to execute is resolved dynamically during program runtime using base class pointers/references.
  - **Function Overriding:** Redefining a base class member function in a derived class using `virtual` functions via inheritance.



2. FUNCTION OVERLOADING & MEMORY MECHANICS

Conceptual Explanation

Function Overloading is a mechanism in C++ where two or more functions share the **exact same function name** but have different parameter lists (different count, data types, or ordering of parameters).

Instead of inventing artificial function names like `addInt()`, `addDouble()`, or `addThreeInts()`, developers define a unified, clean interface using a single function name: `add()`.

How It Works in Memory: Name Mangling (Name Decoration)

CPUs and object-file linkers do not understand overloaded functions with identical names—they require distinct memory symbols for every function entry point. The C++ compiler resolves this during compilation using a mechanism called **Name Mangling (Name Decoration)**:

1. The compiler generates a unique, internal assembly symbol for each overloaded function by encoding its name alongside its parameter types.
2. Each mangled function is assigned its own discrete address in the code/text memory segment.
3. When a call occurs, the compiler maps the call to the corresponding mangled symbol with **zero runtime performance penalty**.

```
[ COMPILER NAME MANGLING IN CODE MEMORY ]

Source Code Prototypes:                Compiler Generated Symbol (Mangled):     RAM Code Address:
int add(int, int);              ⟶     _Z3addii                          ⟶     0x00401000
int add(int, int, int);         ⟶     _Z3addiii                         ⟶     0x00401040
double add(double, double);     ⟶     _Z3adddd                          ⟶     0x00401080
```

3. SYNTAX & RULES FOR FUNCTION OVERLOADING

The 3 Valid Criteria for Overloading

Two or more functions can be overloaded if their parameter signatures differ in at least one of the following ways:

1. **Difference in Number of Parameters:** e.g., `add(int, int)` vs. `add(int, int, int)`.
2. **Difference in Data Types of Parameters:** e.g., `add(int, int)` vs. `add(double, double)`.
3. **Difference in Sequence / Order of Data Types:** e.g., `add(int, double)` vs. `add(double, int)`.

> **Critical Rule: Return Type Alone CANNOT Overload a Function**
>
> Functions **cannot** be overloaded based solely on differing return types (e.g., `int add(int, int)` and `double add(int, int)`). When called as `add(5, 10);`, the compiler cannot determine which function to invoke, triggering a compilation error!

4. HOW FUNCTION OVERLOADING IS RESOLVED (COMPILER STEPS)

When a function call is parsed, the C++ compiler executes a structured 3-step resolution algorithm to find the best viable function candidate:

Step 1: Exact Type Match

The compiler searches for a prototype where the number and types of actual arguments match the formal parameters identically without any conversion. If found, it is invoked immediately.

Step 2: Match Through Type Promotion

If no exact match is found, the compiler applies standard **Integral and Floating-point Promotions**:

• `char`, `unsigned char`, `short` → Promoted to `int`

• `float` → Promoted to `double`

Step 3: Match Through Standard Type Conversion

If no match occurs after promotion, the compiler attempts implicit built-in standard conversions (e.g., `int` to `double`, `double` to `int`, or pointer conversions). If multiple viable candidates exist, compilation fails with an **Ambiguous Call Error**.

5. FUNCTION OVERLOADING VS. FUNCTION OVERRIDING

Feature	Function Overloading	Function Overriding
Polymorphism Type	**Compile-Time Polymorphism** (Static / Early Binding).	**Run-Time Polymorphism** (Dynamic / Late Binding).
Scope	Occurs within the same class or global namespace scope.	Occurs across Base and Derived classes via Inheritance.
Function Signature	Must have **different parameter lists** (count/ types).	Must have the **exact same parameter signature** and return type.
Keywords Required	No special keywords needed.	Requires `virtual` in Base class and `override` in Derived.
Execution Performance	Faster (Direct compile-time resolution, zero overhead).	Slight runtime overhead due to Virtual Table (vtable) lookups.

6. PRACTICAL CODE DEMONSTRATION

```cpp
// Comprehensive Function Overloading Demonstration in C++
#include <iostream>
using namespace std;

// Prototype 1: Two integer arguments
int add(int a, int b) {
    cout << "[Prototype 1: add(int, int)] -> ";
    return a + b;
}

// Prototype 2: Three integer arguments
int add(int a, int b, int c) {
    cout << "[Prototype 2: add(int, int, int)] -> ";
    return a + b + c;
}

// Prototype 3: Two double arguments
double add(double x, double y) {
    cout << "[Prototype 3: add(double, double)] -> ";
    return x + y;
}

// Prototype 4: Mixed types (int, double)
double add(int a, double b) {
    cout << "[Prototype 4: add(int, double)] -> ";
    return a + b;
}

// Prototype 5: Mixed types (double, int)
double add(double a, int b) {
    cout << "[Prototype 5: add(double, int)] -> ";
    return a + b;
}

int main() {
    // 1. Invokes Prototype 1
    cout << "Result: " << add(5, 10) << endl;

    // 2. Invokes Prototype 2
    cout << "Result: " << add(5, 10, 15) << endl;

    // 3. Invokes Prototype 3
    cout << "Result: " << add(12.5, 7.5) << endl;

    // 4. Invokes Prototype 4
    cout << "Result: " << add(15, 10.0) << endl;

    // 5. Invokes Prototype 5
    cout << "Result: " << add(0.75, 5) << endl;

    return 0;
}
```

Best Practice: Avoiding Ambiguity with Default Arguments

Do not combine function overloading with default arguments indiscriminately. Declaring `void compute(int x);` alongside `void compute(int x, int y = 0);` creates ambiguity when invoked as `compute(10);`!